

LAPORAN PRAKTIKUM DEEP LEARNING
CONVOLUTIONAL NEURAL NETWORK UNTUK
KLASIFIKASI CITRA



Disusun oleh:

HABIBI PUTRA RIZQULLAH

NIM 2411531001

Kelas B

Dosen Pengampu:

Meza Silvana, S.T., M.T.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026**

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Praktikum	1
BAB II LANDASAN TEORI	3
2.1 Deep Learning	3
2.2 Convolutional Neural Network	3
2.3 Komponen CNN	3
2.4 Dataset MNIST	3
BAB III METODOLOGI DAN LANGKAH PRAKTIKUM	4
3.1 Alat dan Bahan	4
3.2 Langkah Praktikum	4
3.2.1 Mengimpor Library dan Menentukan Perangkat	4
3.2.2 Memuat Dataset MNIST	5
3.2.3 Menampilkan Contoh Citra	6
3.2.4 Mendefinisikan Model CNN	6
3.2.5 Menentukan Loss dan Optimizer	7
3.2.6 Membuat Fungsi Training dan Evaluasi	7
3.2.7 Melatih Model	8
3.2.8 Menampilkan Grafik	9
3.2.9 Mengevaluasi Hasil Akhir	9
3.2.10 Menampilkan Hasil Prediksi	10
3.3 Tugas Individu	11
3.3.1 Menyiapkan Library dan Dataset	11
3.3.2 Mendefinisikan CNN 1	12
3.3.3 Mendefinisikan CNN 2	13
3.3.4 Memeriksa Bentuk Output	14
3.3.5 Membuat Fungsi Evaluasi	14
3.3.6 Membuat Fungsi Eksperimen	15
3.3.7 Menjalankan Kedua Model	16
3.3.8 Menyusun Tabel Hasil	16
3.3.9 Membandingkan Grafik	17
3.3.10 Menyusun Kesimpulan Eksperimen	18
3.3.11 Menampilkan Prediksi CNN 2	19
BAB IV HASIL DAN PEMBAHASAN	20
4.1 Hasil Praktikum	20
4.2 Pembahasan	20
4.2.1 Analisis Overfitting	20
4.2.2 Perbandingan Model	21
4.3 Jawaban Pertanyaan Reflektif	21
4.3.1 Mengapa CNN Lebih Baik Dibanding MLP untuk Citra?	21
4.3.2 Apa Fungsi Pooling Layer dalam CNN?	21
4.3.3 Bagaimana Pengaruh Ukuran Kernel terhadap Hasil Konvolusi?	22
BAB V PENUTUP	23
5.1 Kesimpulan	23
DAFTAR PUSTAKA	24

DAFTAR GAMBAR

Gambar 3.1 Import library dan penentuan perangkat.....	5
Gambar 3.2 Memuat dataset MNIST	5
Gambar 3.3 Contoh citra pada dataset MNIST	6
Gambar 3.4 Arsitektur model CNN	7
Gambar 3.5 Penentuan loss dan optimizer	7
Gambar 3.6 Fungsi training dan evaluasi	8
Gambar 3.7 Proses training model selama lima epoch.....	8
Gambar 3.8 Kode visualisasi loss dan akurasi	9
Gambar 3.9 Grafik loss dan akurasi	9
Gambar 3.10 Evaluasi hasil akhir model.....	10
Gambar 3.11 Kode prediksi citra uji	10
Gambar 3.12 Hasil prediksi citra uji	11
Gambar 3.13 Import library untuk tugas individu.....	12
Gambar 3.14 Persiapan dataset tugas individu.....	12
Gambar 3.15 Arsitektur CNN 1	13
Gambar 3.16 Arsitektur CNN 2	13
Gambar 3.17 Pemeriksaan bentuk output model.....	14
Gambar 3.18 Fungsi evaluasi.....	14
Gambar 3.19 Fungsi eksperimen bagian pertama	15
Gambar 3.20 Fungsi eksperimen bagian kedua.....	15
Gambar 3.21 Hasil training CNN 1 dan CNN 2.....	16
Gambar 3.22 Tabel hasil eksperimen	17
Gambar 3.23 Kode grafik perbandingan	17
Gambar 3.24 Grafik akurasi dan loss testing.....	18
Gambar 3.25 Kode analisis hasil eksperimen.....	18
Gambar 3.26 Kesimpulan hasil eksperimen	18
Gambar 3.27 Kode prediksi CNN 2	19
Gambar 3.28 Hasil prediksi CNN 2	19

DAFTAR TABEL

Tabel 4.1 Nilai loss dan akurasi setiap epoch.....	20
Tabel 4.2 Perbandingan CNN 1 dan CNN 2	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengolahan citra merupakan salah satu penerapan deep learning yang banyak digunakan. Berbeda dengan data berbentuk tabel, citra memiliki hubungan spasial antarpiksel yang perlu dipertahankan selama proses pembelajaran. Convolutional Neural Network atau CNN dirancang untuk mempelajari pola lokal seperti garis, bentuk, dan tekstur melalui operasi konvolusi.

Pada praktikum ini, CNN digunakan untuk mengklasifikasikan angka tulisan tangan pada dataset MNIST. Model dibangun menggunakan PyTorch dan dilatih selama lima epoch. Performa model diamati melalui loss, akurasi training, akurasi testing, grafik pembelajaran, serta contoh hasil prediksi.

Tugas individu dilakukan dengan membandingkan CNN 1 yang memiliki dua lapisan konvolusi dengan CNN 2 yang memiliki tiga lapisan konvolusi dan kernel awal 5×5 . Perbandingan ini digunakan untuk melihat pengaruh kedalaman jaringan terhadap akurasi dan waktu training.

1.2 Rumusan Masalah

1. Bagaimana menerapkan CNN untuk klasifikasi citra MNIST menggunakan PyTorch?
2. Bagaimana perubahan loss dan akurasi model selama training?
3. Apakah model menunjukkan gejala overfitting?
4. Bagaimana pengaruh penambahan lapisan konvolusi terhadap akurasi dan waktu training?

1.3 Tujuan Praktikum

1. Memahami cara kerja CNN pada klasifikasi citra.
2. Menerapkan konvolusi, ReLU, pooling, dan fully connected menggunakan PyTorch.
3. Menganalisis loss, akurasi training, akurasi testing, dan hasil prediksi.
4. Membandingkan performa jaringan dengan kedalaman yang berbeda.

BAB II

LANDASAN TEORI

2.1 Deep Learning

Deep learning adalah bagian dari machine learning yang menggunakan jaringan saraf berlapis untuk mempelajari representasi data. Lapisan awal mempelajari pola sederhana, sedangkan lapisan berikutnya menggabungkan pola tersebut menjadi fitur yang lebih kompleks [1].

2.2 Convolutional Neural Network

CNN merupakan jaringan saraf yang menggunakan filter atau kernel untuk mengekstraksi fitur dari citra. Filter digeser pada seluruh bagian citra dan menghasilkan feature map. Penggunaan bobot yang sama pada berbagai posisi membuat CNN memiliki parameter yang lebih efisien dibandingkan jaringan fully connected untuk data citra [2].

2.3 Komponen CNN

Lapisan konvolusi digunakan untuk mencari fitur lokal. ReLU mengubah nilai negatif menjadi nol. Max pooling mengurangi ukuran feature map dengan memilih nilai terbesar pada suatu area. Setelah ekstraksi fitur selesai, flatten mengubah feature map menjadi vektor yang kemudian diproses oleh fully connected layer.

2.4 Dataset MNIST

MNIST terdiri atas 60.000 citra training dan 10.000 citra testing. Setiap citra berupa gambar berskala abu-abu dengan ukuran 28 x 28 piksel dan memiliki label angka 0 sampai 9. Dataset ini sering digunakan untuk menguji model klasifikasi citra karena bentuk datanya sederhana dan mudah dipahami.

BAB III

METODOLOGI DAN LANGKAH PRAKTIKUM

3.1 Alat dan Bahan

1. Laptop atau komputer yang terhubung ke internet.
2. Google Colaboratory sebagai lingkungan pemrograman.
3. Python, PyTorch, torchvision, NumPy, Matplotlib, pandas, dan time.
4. Dataset MNIST.
5. Notebook Praktikum2_DeepLearning.ipynb dan Tugas_Praktikum2_DeepLearning.ipynb.

3.2 Langkah Praktikum

Tahapan praktikum dilakukan mulai dari menyiapkan library sampai menampilkan hasil prediksi. Setiap bagian dijelaskan sesuai urutan sel pada notebook.

3.2.1 Mengimpor Library dan Menentukan Perangkat

Library yang dibutuhkan diimpor terlebih dahulu. Seed bernilai 42 digunakan agar hasil lebih konsisten. Program menggunakan GPU jika tersedia dan memakai CPU jika GPU tidak tersedia.

```

import random
import time

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Perangkat yang digunakan: {device}")

Perangkat yang digunakan: cpu

```

Gambar 3.1 Import library dan penentuan perangkat

3.2.2 Memuat Dataset MNIST

Citra diubah menjadi tensor dan dinormalisasi. Data training dan testing dimuat dengan batch berukuran 64. Hasilnya menunjukkan 60.000 data training dan 10.000 data testing.

```

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

trainset = torchvision.datasets.MNIST(
    root="./data", train=True, download=True, transform=transform
)
testset = torchvision.datasets.MNIST(
    root="./data", train=False, download=True, transform=transform
)

trainloader = DataLoader(
    trainset, batch_size=64, shuffle=True, num_workers=0,
    pin_memory=torch.cuda.is_available()
)
testloader = DataLoader(
    testset, batch_size=64, shuffle=False, num_workers=0,
    pin_memory=torch.cuda.is_available()
)

print(f"Jumlah data latih : {len(trainset):,}")
print(f"Jumlah data uji   : {len(testset):,}")
print(f"Ukuran satu gambar: {trainset[0][0].shape}")

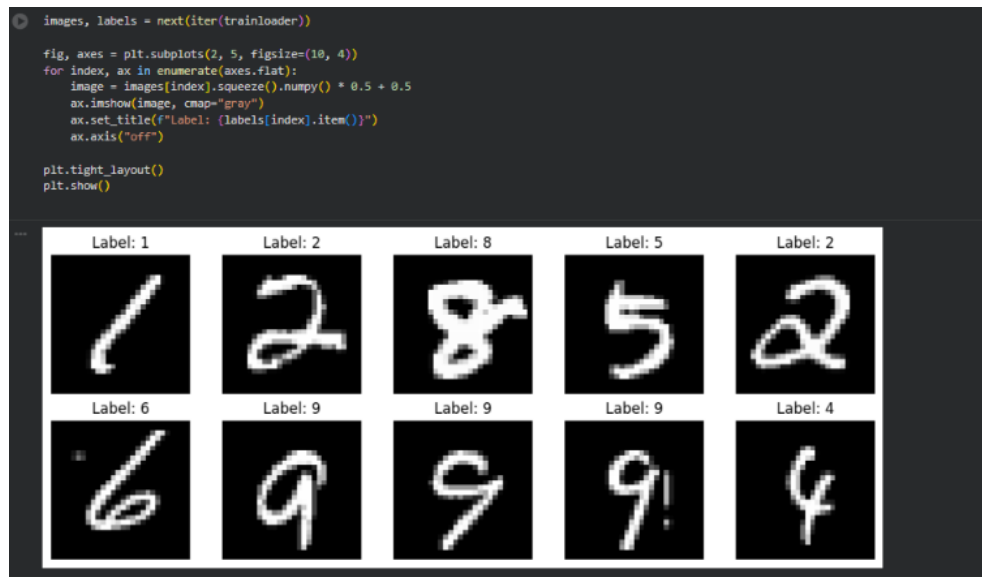
100% |#####| 9.91M/9.91M [00:00<00:00, 15.3MB/s]
100% |#####| 28.9k/28.9k [00:00<00:00, 496kB/s]
100% |#####| 1.65M/1.65M [00:00<00:00, 4.39MB/s]
100% |#####| 4.54k/4.54k [00:00<00:00, 6.50MB/s]Jumlah data latih : 60,000
Jumlah data uji   : 10,000
Ukuran satu gambar: torch.Size([1, 28, 28])

```

Gambar 3.2 Memuat dataset MNIST

3.2.3 Menampilkan Contoh Citra

Satu batch diambil dari trainloader dan sepuluh citra pertama ditampilkan untuk memeriksa kesesuaian gambar dengan label.



Gambar 3.3 Contoh citra pada dataset MNIST

3.2.4 Mendefinisikan Model CNN

Model terdiri atas dua lapisan konvolusi. Setiap lapisan konvolusi diikuti ReLU dan max pooling. Hasil ekstraksi fitur diratakan sebelum masuk ke dua lapisan linear.

```

class CNNModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)

model = CNNModel().to(device)
print(model)

CNNModel(
  (features): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1568, out_features=128, bias=True)
    (2): ReLU()
    (3): Linear(in_features=128, out_features=10, bias=True)
  )
)

```

Gambar 3.4 Arsitektur model CNN

3.2.5 Menentukan Loss dan Optimizer

CrossEntropyLoss digunakan untuk menghitung kesalahan klasifikasi, sedangkan Adam dengan learning rate 0,001 digunakan untuk memperbarui bobot model.

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

Gambar 3.5 Penentuan loss dan optimizer

3.2.6 Membuat Fungsi Training dan Evaluasi

Fungsi train_one_epoch melakukan forward pass, perhitungan loss, backpropagation, dan pembaruan bobot. Fungsi evaluate menghitung loss dan akurasi tanpa memperbarui bobot.

```

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * images.size(0)
        predictions = outputs.argmax(dim=1)
        correct += (predictions == labels).sum().item()
        total += labels.size(0)

    return total_loss / total, 100 * correct / total

def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)
            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item() * images.size(0)
            predictions = outputs.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

    return total_loss / total, 100 * correct / total

```

Gambar 3.6 Fungsi training dan evaluasi

3.2.7 Melatih Model

Model dilatih selama lima epoch. Nilai loss dan akurasi training serta testing disimpan untuk dianalisis. Waktu training yang diperoleh adalah 267,38 detik.

```

epochs = 5
history = {
    "train_loss": [], "test_loss": [],
    "train_accuracy": [], "test_accuracy": []
}

start_time = time.perf_counter()

for epoch in range(epochs):
    train_loss, train_accuracy = train_one_epoch(
        model, trainloader, criterion, optimizer, device
    )
    test_loss, test_accuracy = evaluate(
        model, testloader, criterion, device
    )

    history["train_loss"].append(train_loss)
    history["test_loss"].append(test_loss)
    history["train_accuracy"].append(train_accuracy)
    history["test_accuracy"].append(test_accuracy)

    print(
        f"Epoch (epoch + 1)/(epochs) | "
        f"train loss: (train_loss:.4f) | "
        f"train acc: (train_accuracy:.2f)% | "
        f"test loss: (test_loss:.4f) | "
        f"test acc: (test_accuracy:.2f)% "
    )

training_time = time.perf_counter() - start_time
print()
print(f"Waktu training: (training_time:.2f) detik")

```

Epoch 1/5	train loss: 0.1995	train acc: 93.99%	test loss: 0.8624	test acc: 97.96%
Epoch 2/5	train loss: 0.0572	train acc: 98.23%	test loss: 0.0480	test acc: 98.37%
Epoch 3/5	train loss: 0.0390	train acc: 98.80%	test loss: 0.0383	test acc: 98.78%
Epoch 4/5	train loss: 0.0293	train acc: 99.09%	test loss: 0.0312	test acc: 99.03%
Epoch 5/5	train loss: 0.0234	train acc: 99.26%	test loss: 0.0313	test acc: 99.00%

Waktu training: 267.38 detik

Gambar 3.7 Proses training model selama lima epoch

3.2.8 Menampilkan Grafik

Riwayat loss dan akurasi divisualisasikan agar perubahan performa setiap epoch dapat diamati dengan lebih mudah.

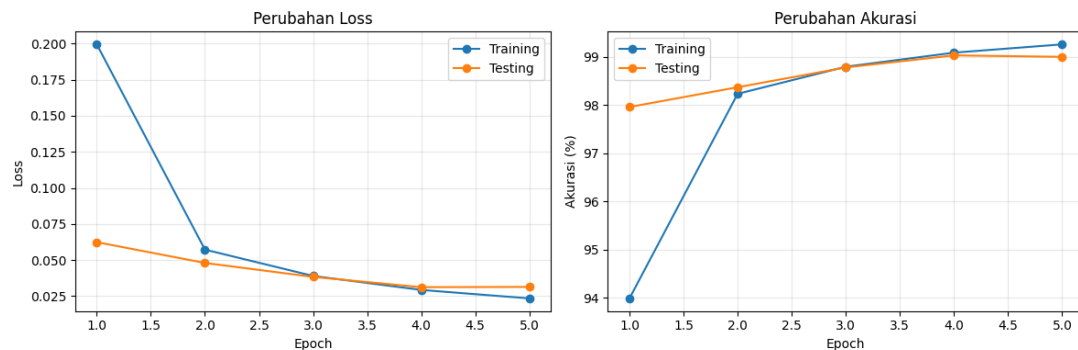
```
epoch_numbers = range(1, epochs + 1)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(epoch_numbers, history["train_loss"], marker="o", label="Training")
axes[0].plot(epoch_numbers, history["test_loss"], marker="o", label="Testing")
axes[0].set_title("Perubahan Loss")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot(epoch_numbers, history["train_accuracy"], marker="o", label="Training")
axes[1].plot(epoch_numbers, history["test_accuracy"], marker="o", label="Testing")
axes[1].set_title("Perubahan Akurasi")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Akurasi (%)")
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

Gambar 3.8 Kode visualisasi loss dan akurasi



Gambar 3.9 Grafik loss dan akurasi

3.2.9 Mengevaluasi Hasil Akhir

Akurasi training akhir sebesar 99,26 persen dan akurasi testing sebesar 99,00 persen. Selisih 0,26 poin persentase menunjukkan belum terlihat overfitting yang kuat.

```
final_train_accuracy = history["train_accuracy"][-1]
final_test_accuracy = history["test_accuracy"][-1]
accuracy_gap = final_train_accuracy - final_test_accuracy

print(f"Akurasi training akhir : {final_train_accuracy:.2f}%")
print(f"Akurasi testing akhir : {final_test_accuracy:.2f}%")
print(f"Selisih akurasi : {accuracy_gap:.2f} poin persentase")

if accuracy_gap > 5:
    print("Indikasi: mulai terlihat overfitting karena selisih akurasi cukup besar.")
else:
    print("Indikasi: belum terlihat overfitting yang kuat dari selisih akurasi akhir.")

--- Akurasi training akhir : 99.26%
    Akurasi testing akhir : 99.00%
    Selisih akurasi : 0.26 poin persentase
    Indikasi: belum terlihat overfitting yang kuat dari selisih akurasi akhir.
```

Gambar 3.10 Evaluasi hasil akhir model

3.2.10 Menampilkan Hasil Prediksi

Model digunakan untuk memprediksi sepuluh citra uji. Warna hijau pada judul gambar menunjukkan prediksi sesuai dengan label sebenarnya.

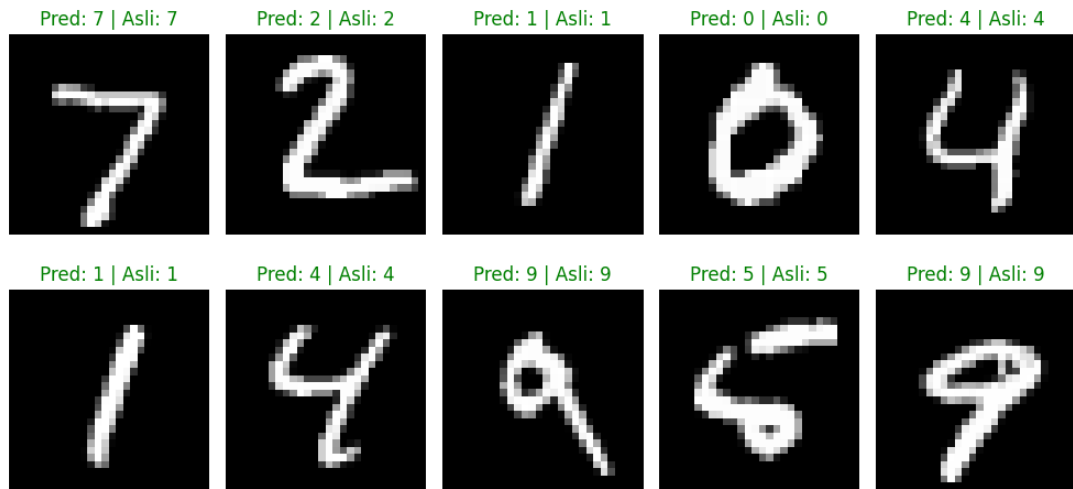
```
model.eval()
test_images, test_labels = next(iter(testloader))

with torch.no_grad():
    outputs = model(test_images.to(device))
    predictions = outputs.argmax(dim=1).cpu()

fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for index, ax in enumerate(axes.flat):
    image = test_images[index].squeeze().numpy() * 0.5 + 0.5
    is_correct = predictions[index].item() == test_labels[index].item()
    ax.imshow(image, cmap="gray")
    ax.set_title(
        f"Pred: {predictions[index].item()} | Asli: {test_labels[index].item()}",
        color="green" if is_correct else "red"
    )
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Gambar 3.11 Kode prediksi citra uji



Gambar 3.12 Hasil prediksi citra uji

3.3 Tugas Individu

Tugas individu membandingkan CNN 1 sebagai baseline dengan CNN 2 yang memiliki lapisan konvolusi tambahan. Seluruh model dilatih dengan dataset, jumlah epoch, dan learning rate yang sama.

3.3.1 Menyiapkan Library dan Dataset

Library, seed, perangkat, dataset, serta parameter eksperimen disiapkan. Kedua model menggunakan lima epoch dan learning rate 0,001.

```
import copy
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Perangkat yang digunakan: {device}")

*** Perangkat yang digunakan: cpu
```

Gambar 3.13 Import library untuk tugas individu

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

trainset = torchvision.datasets.MNIST(
    root="./data", train=True, download=True, transform=transform
)
testset = torchvision.datasets.MNIST(
    root="./data", train=False, download=True, transform=transform
)

trainloader = DataLoader(
    trainset, batch_size=64, shuffle=True, num_workers=0,
    pin_memory=torch.cuda.is_available()
)
testloader = DataLoader(
    testset, batch_size=64, shuffle=False, num_workers=0,
    pin_memory=torch.cuda.is_available()
)

EPOCHS = 5
LEARNING_RATE = 0.001

print(f"Data latih: {len(trainset):,}")
print(f"Data uji : {len(testset):,}")

Data latih: 60,000
Data uji : 10,000
```

Gambar 3.14 Persiapan dataset tugas individu

3.3.2 Mendefinisikan CNN 1

CNN 1 menggunakan dua lapisan konvolusi dengan kernel 3 x 3 dan digunakan sebagai baseline.

```

class CNNBaseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        return self.classifier(self.features(x))

```

Gambar 3.15 Arsitektur CNN 1

3.3.3 Mendefinisikan CNN 2

CNN 2 menggunakan tiga lapisan konvolusi. Lapisan pertama memakai kernel 5 x 5, kemudian ditambahkan dropout 0,25 pada classifier.

```

class CNNDeeper(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 3 * 3, 128),
            nn.ReLU(),
            nn.Dropout(0.25),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        return self.classifier(self.features(x))

```

Gambar 3.16 Arsitektur CNN 2

3.3.4 Memeriksa Bentuk Output

Dummy input digunakan untuk memastikan kedua model menghasilkan output berukuran 4 x 10 sesuai jumlah data dalam batch dan sepuluh kelas angka.

```
dummy_input = torch.randn(4, 1, 28, 28).to(device)

baseline_check = CNNBaseline().to(device)
deeper_check = CNNDeeper().to(device)

print("Output CNN 1:", baseline_check(dummy_input).shape)
print("Output CNN 2:", deeper_check(dummy_input).shape)

del baseline_check, deeper_check, dummy_input
if torch.cuda.is_available():
    torch.cuda.empty_cache()

Output CNN 1: torch.Size([4, 10])
Output CNN 2: torch.Size([4, 10])
```

Gambar 3.17 Pemeriksaan bentuk output model

3.3.5 Membuat Fungsi Evaluasi

Fungsi evaluasi menghitung rata-rata loss dan akurasi pada data yang diberikan tanpa menjalankan backpropagation.

```
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)
            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(dim=1) == labels).sum().item()
            total += labels.size(0)

    return total_loss / total, 100 * correct / total
```

Gambar 3.18 Fungsi evaluasi

3.3.6 Membuat Fungsi Eksperimen

Fungsi `run_experiment` menyatukan proses training, evaluasi, penyimpanan model terbaik, dan pencatatan waktu. Fungsi yang sama dipakai agar perbandingan kedua model tetap adil.

```
def run_experiment(model, model_name, trainloader, testloader, epochs, device):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

    history = {
        "train_loss": [], "test_loss": [],
        "train_accuracy": [], "test_accuracy": []
    }
    best_accuracy = -1.0
    best_state = None
    start_time = time.perf_counter()

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in trainloader:
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(dim=1) == labels.sum().item()).sum().item()
            total += labels.size(0)

        train_loss = running_loss / total
        train_accuracy = 100 * correct / total
        test_loss, test_accuracy = evaluate(model, testloader, criterion, device)

        history["train_loss"].append(train_loss)
```

Gambar 3.19 Fungsi eksperimen bagian pertama

```
        history["test_loss"].append(test_loss)
        history["train_accuracy"].append(train_accuracy)
        history["test_accuracy"].append(test_accuracy)

        if test_accuracy > best_accuracy:
            best_accuracy = test_accuracy
            best_state = copy.deepcopy(model.state_dict())

        print(
            f"{model_name} | Epoch {epoch + 1}/{epochs} | "
            f"train acc: {train_accuracy:.2f}% | test acc: {test_accuracy:.2f}%"
        )

    training_time = time.perf_counter() - start_time
    model.load_state_dict(best_state)
    final_train_loss, final_train_accuracy = evaluate(
        model, trainloader, criterion, device
    )
    final_test_loss, final_test_accuracy = evaluate(
        model, testloader, criterion, device
    )

    result = {
        "model": model,
        "history": history,
        "train_accuracy": final_train_accuracy,
        "test_accuracy": final_test_accuracy,
        "training_time": training_time
    }
    return result
```

Gambar 3.20 Fungsi eksperimen bagian kedua

3.3.7 Menjalankan Kedua Model

CNN 1 dan CNN 2 dilatih selama lima epoch. CNN 2 memperoleh akurasi testing yang lebih tinggi pada setiap epoch.

```
torch.manual_seed(SEED)
baseline_result = run_experiment(
    CNNBaseline(), "CNN 1", trainloader, testloader, EPOCHS, device
)

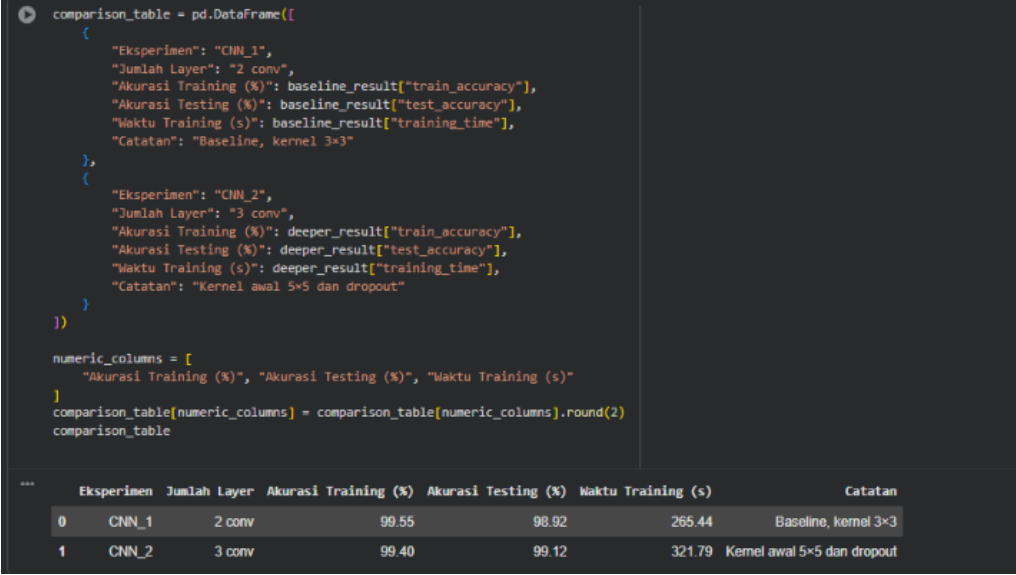
torch.manual_seed(SEED)
deeper_result = run_experiment(
    CNNDeeper(), "CNN 2", trainloader, testloader, EPOCHS, device
)
```

***	CNN 1	Epoch 1/5	train acc: 93.68%	test acc: 98.08%
	CNN 1	Epoch 2/5	train acc: 98.22%	test acc: 98.64%
	CNN 1	Epoch 3/5	train acc: 98.69%	test acc: 98.72%
	CNN 1	Epoch 4/5	train acc: 99.00%	test acc: 98.74%
	CNN 1	Epoch 5/5	train acc: 99.23%	test acc: 98.92%
	CNN 2	Epoch 1/5	train acc: 92.39%	test acc: 98.63%
	CNN 2	Epoch 2/5	train acc: 98.11%	test acc: 98.96%
	CNN 2	Epoch 3/5	train acc: 98.62%	test acc: 98.99%
	CNN 2	Epoch 4/5	train acc: 98.86%	test acc: 99.12%
	CNN 2	Epoch 5/5	train acc: 99.05%	test acc: 99.12%

Gambar 3.21 Hasil training CNN 1 dan CNN 2

3.3.8 Menyusun Tabel Hasil

Hasil akhir kedua eksperimen disusun menjadi tabel yang berisi jumlah layer, akurasi training, akurasi testing, waktu training, dan catatan arsitektur.



```

comparison_table = pd.DataFrame([
    {
        "Eksperimen": "CNN_1",
        "Jumlah Layer": "2 conv",
        "Akurasi Training (%)": baseline_result["train_accuracy"],
        "Akurasi Testing (%)": baseline_result["test_accuracy"],
        "Waktu Training (s)": baseline_result["training_time"],
        "Catatan": "Baseline, kernel 3x3"
    },
    {
        "Eksperimen": "CNN_2",
        "Jumlah Layer": "3 conv",
        "Akurasi Training (%)": deeper_result["train_accuracy"],
        "Akurasi Testing (%)": deeper_result["test_accuracy"],
        "Waktu Training (s)": deeper_result["training_time"],
        "Catatan": "Kernel awal 5x5 dan dropout"
    }
])

numeric_columns = [
    "Akurasi Training (%)", "Akurasi Testing (%)", "Waktu Training (s)"
]
comparison_table[numeric_columns] = comparison_table[numeric_columns].round(2)
comparison_table

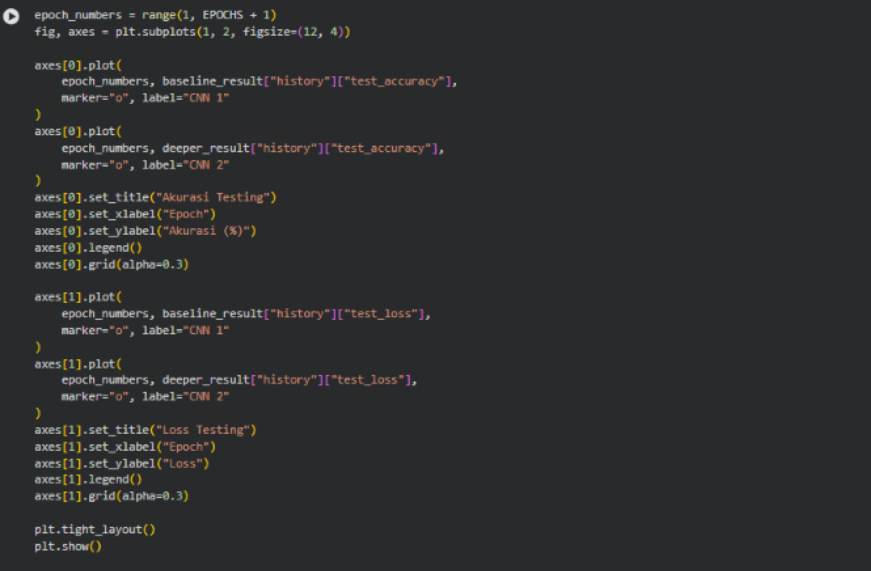
```

	Eksperimen	Jumlah Layer	Akurasi Training (%)	Akurasi Testing (%)	Waktu Training (s)	Catatan
0	CNN_1	2 conv	99.55	98.92	265.44	Baseline, kernel 3x3
1	CNN_2	3 conv	99.40	99.12	321.79	Kernel awal 5x5 dan dropout

Gambar 3.22 Tabel hasil eksperimen

3.3.9 Membandingkan Grafik

Grafik pertama membandingkan akurasi testing, sedangkan grafik kedua membandingkan loss testing CNN 1 dan CNN 2.



```

epoch_numbers = range(1, EPOCHS + 1)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

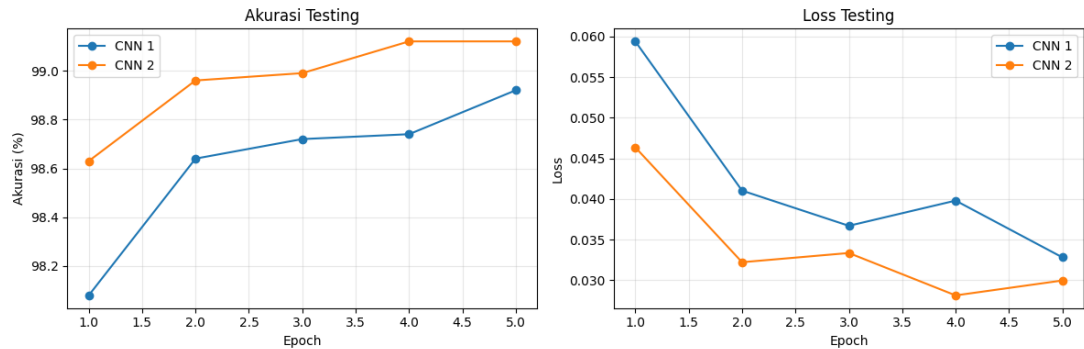
axes[0].plot(
    epoch_numbers, baseline_result["history"]["test_accuracy"],
    marker="o", label="CNN 1"
)
axes[0].plot(
    epoch_numbers, deeper_result["history"]["test_accuracy"],
    marker="o", label="CNN 2"
)
axes[0].set_title("Akurasi Testing")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Akurasi (%)")
axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot(
    epoch_numbers, baseline_result["history"]["test_loss"],
    marker="o", label="CNN 1"
)
axes[1].plot(
    epoch_numbers, deeper_result["history"]["test_loss"],
    marker="o", label="CNN 2"
)
axes[1].set_title("Loss Testing")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

```

Gambar 3.23 Kode grafik perbandingan



Gambar 3.24 Grafik akurasi dan loss testing

3.3.10 Menyusun Kesimpulan Eksperimen

Selisih akurasi, waktu training, serta gap training dan testing dihitung untuk menilai pengaruh penambahan lapisan.

```
accuracy_difference = (
    deeper_result["test_accuracy"] - baseline_result["test_accuracy"]
)
time_difference = (
    deeper_result["training_time"] - baseline_result["training_time"]
)
baseline_gap = (
    baseline_result["train_accuracy"] - baseline_result["test_accuracy"]
)
deeper_gap = (
    deeper_result["train_accuracy"] - deeper_result["test_accuracy"]
)

if accuracy_difference > 0:
    accuracy_statement = (
        f"CNN 2 lebih akurat {accuracy_difference:.2f} poin persentase daripada CNN 1."
    )
elif accuracy_difference < 0:
    accuracy_statement = (
        f"CNN 2 lebih rendah {abs(accuracy_difference):.2f} poin persentase daripada CNN 1."
    )
else:
    accuracy_statement = "Kedua model menghasilkan akurasi testing yang sama."

if time_difference > 0:
    time_statement = f"CNN 2 membutuhkan waktu {time_difference:.2f} detik lebih lama."
else:
    time_statement = f"CNN 2 selesai {abs(time_difference):.2f} detik lebih cepat."

print("Kesimpulan:")
print(accuracy_statement)
print(time_statement)
print(f"Gap train-test CNN 1: {baseline_gap:.2f} poin persentase.")
print(f"Gap train-test CNN 2: {deeper_gap:.2f} poin persentase.")
print(
    "Penambahan lapisan memberi kapasitas ekstraksi fitur yang lebih besar, "
    "tetapi juga menambah beban komputasi. Karena itu, model yang lebih dalam "
    "tidak otomatis menjadi pilihan paling efisien."
)
```

Gambar 3.25 Kode analisis hasil eksperimen

```
--- Kesimpulan:
CNN 2 lebih akurat 0.20 poin persentase daripada CNN 1.
CNN 2 membutuhkan waktu 56.36 detik lebih lama.
Gap train-test CNN 1: 0.63 poin persentase.
Gap train-test CNN 2: 0.28 poin persentase.
Penambahan lapisan memberi kapasitas ekstraksi fitur yang lebih besar, tetapi juga menambah beban komputasi. Karena itu, model yang lebih dalam tidak otomatis menjadi pilihan paling efisien.
```

Gambar 3.26 Kesimpulan hasil eksperimen

3.3.11 Menampilkan Prediksi CNN 2

Model CNN 2 digunakan untuk memprediksi sepuluh citra testing. Seluruh contoh yang ditampilkan memperoleh prediksi yang sesuai dengan label sebenarnya.

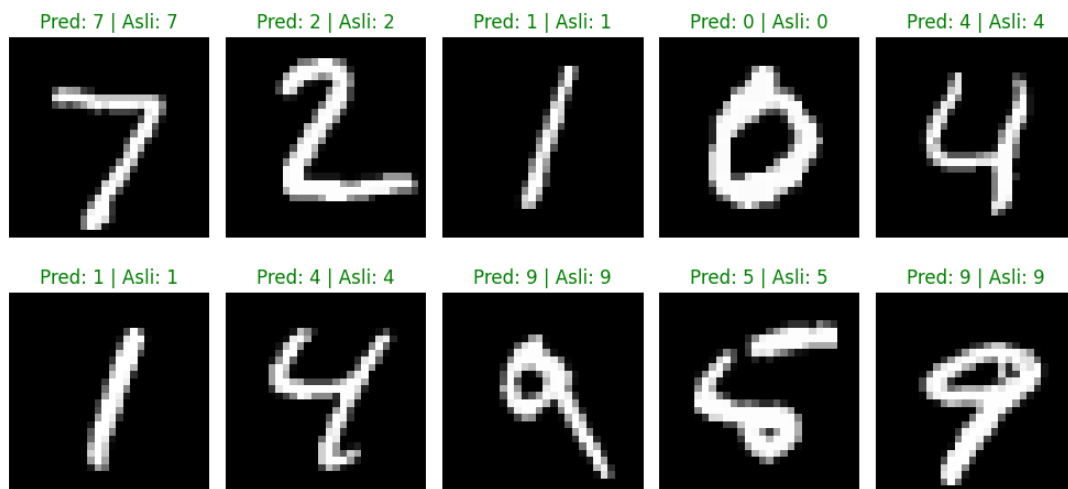
```
model = deeper_result["model"]
model.eval()
images, labels = next(iter(testloader))

with torch.no_grad():
    predictions = model(images.to(device)).argmax(dim=1).cpu()

fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for index, ax in enumerate(axes.flat):
    image = images[index].squeeze().numpy() * 0.5 + 0.5
    is_correct = predictions[index].item() == labels[index].item()
    ax.imshow(image, cmap="gray")
    ax.set_title(
        f"Pred: {predictions[index].item()} | Asli: {labels[index].item()}",
        color="green" if is_correct else "red"
    )
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Gambar 3.27 Kode prediksi CNN 2



Gambar 3.28 Hasil prediksi CNN 2

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Hasil training menunjukkan loss menurun dan akurasi meningkat selama lima epoch. Rincian hasil praktikum ditampilkan pada Tabel 4.1.

Epoch	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
1	0,1995	93,99	0,0624	97,96
2	0,0572	98,23	0,0480	98,37
3	0,0390	98,80	0,0383	98,78
4	0,0293	99,09	0,0312	99,03
5	0,0234	99,26	0,0313	99,00

Tabel 4.1 Nilai loss dan akurasi setiap epoch

Pada epoch pertama, train loss masih sebesar 0,1995 dengan akurasi training 93,99 persen. Pada epoch kelima, train loss turun menjadi 0,0234 dan akurasi training mencapai 99,26 persen. Akurasi testing akhir sebesar 99,00 persen.

4.2 Pembahasan

4.2.1 Analisis Overfitting

Selisih akurasi training dan testing hanya 0,26 poin persentase. Grafik loss training dan testing juga menurun dengan pola yang hampir sama. Test loss naik sangat kecil dari 0,0312 menjadi 0,0313 pada epoch terakhir, tetapi akurasi testing tetap berada di sekitar 99 persen. Berdasarkan lima epoch yang dijalankan, model belum menunjukkan overfitting yang kuat.

4.2.2 Perbandingan Model

Eksperimen	Jumlah Layer	Train Acc (%)	Test Acc (%)	Waktu (s)	Catatan
CNN 1	2 conv	99,55	98,92	265,44	Baseline, kernel 3 x 3
CNN 2	3 conv	99,40	99,12	321,79	Kernel 5 x 5 dan dropout

Tabel 4.2 Perbandingan CNN 1 dan CNN 2

CNN 2 memperoleh akurasi testing 99,12 persen, lebih tinggi 0,20 poin persentase dibandingkan CNN 1. Gap training dan testing CNN 2 sebesar 0,28 poin persentase, sedangkan CNN 1 sebesar 0,63 poin persentase. CNN 2 terlihat lebih stabil pada data testing.

Waktu training CNN 2 lebih lama sekitar 56,36 detik. Penambahan lapisan meningkatkan kemampuan ekstraksi fitur, tetapi juga menambah beban komputasi. Pada dataset MNIST, peningkatan akurasi yang diperoleh cukup kecil dibandingkan tambahan waktu training.

4.3 Jawaban Pertanyaan Reflektif

4.3.1 Mengapa CNN Lebih Baik Dibanding MLP untuk Citra?

CNN mempertahankan hubungan spasial antarpiksel dan menggunakan filter yang sama pada berbagai posisi citra. Hal ini membuat CNN dapat mengenali pola seperti garis dan bentuk dengan parameter yang lebih sedikit. MLP meratakan citra sejak awal sehingga hubungan spasial tidak dimanfaatkan secara langsung.

4.3.2 Apa Fungsi Pooling Layer dalam CNN?

Pooling mengurangi ukuran feature map dengan mengambil nilai yang mewakili suatu area. Max pooling memilih nilai terbesar sehingga fitur utama tetap dipertahankan. Proses ini mengurangi beban komputasi dan membantu model lebih tahan terhadap perubahan kecil pada posisi objek.

4.3.3 Bagaimana Pengaruh Ukuran Kernel terhadap Hasil Konvolusi?

Kernel 3 x 3 menangkap pola lokal dengan jumlah parameter yang lebih sedikit. Kernel 5 x 5 memiliki cakupan yang lebih luas sehingga dapat menangkap konteks lebih besar dalam satu lapisan, tetapi memerlukan perhitungan lebih banyak. Ukuran kernel perlu disesuaikan dengan ukuran citra dan kebutuhan model.

BAB V

PENUTUP

5.1 Kesimpulan

Model CNN dengan dua lapisan konvolusi berhasil mengklasifikasikan citra MNIST dengan akurasi training 99,26 persen dan akurasi testing 99,00 persen. Nilai loss menurun secara konsisten dan selisih akurasi akhir hanya 0,26 poin persentase, sehingga belum terlihat overfitting yang kuat.

CNN 2 yang memiliki tiga lapisan konvolusi memperoleh akurasi testing 99,12 persen, lebih tinggi daripada CNN 1 yang memperoleh 98,92 persen. Namun, waktu training bertambah dari 265,44 detik menjadi 321,79 detik. Penambahan kedalaman jaringan dapat meningkatkan akurasi dan kestabilan, tetapi efisiensi komputasi tetap perlu dipertimbangkan.

DAFTAR PUSTAKA

- [1] Goodfellow, I., Bengio, Y., dan Courville, A. 2016. Deep Learning. Cambridge: MIT Press.
- [2] LeCun, Y., Bottou, L., Bengio, Y., dan Haffner, P. 1998. Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 86(11), 2278-2324.
- [3] Paszke, A., dkk. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems, 32.
- [4] PyTorch. 2026. PyTorch Documentation. <https://pytorch.org/docs/stable/>. Diakses 24 September 2026.
- [5] Torchvision. 2026. MNIST Dataset Documentation. <https://pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html>. Diakses 24 September 2026.